

Conversation-quality rubric — LLM-judge specification (v2)

This file is the **canonical, versioned judge specification** for the benchmark's quality evals. The judge prompt is assembled from this document — it lives in git so every scored conversation can be traced to the exact rubric text that produced it. (v1 asked a judge for scalar sub-scores directly; v2 decomposes every dimension into **binary, evidence-forced checks** *and* derives* the numbers — the judge never picks a number.)

Design principles

1. **Binary checks, not scalars.** Each rubric dimension is a fixed set of pass/fail checks. A check that passes **MUST** cite a short verbatim quote from the transcript as evidence. No quote → no credit. Scores are computed from the booleans by the fixed point mapping below (in `eval-merge.js`), never emitted by the judge. 2. **Blind judging.** Batches are anonymized before judging: the judge sees an opaque key, with vendor and store names stripped from metadata and masked in transcript text. Judges score behavior, not brands. 3. **Deterministic signals beside the judge.** What code can measure, code measures: price mentions, links, review/rating mentions, option counts are detected by regex at pack time and **enforced at merge time** — a rich-element check cannot pass if the deterministic signal is absent from the transcript. 4. **Outcome correctness, not outcome preference.** A handover is not automatically a failure: escalating when the answer genuinely isn't available — after collecting the right info and setting expectations — is correct support behavior and scores as such. (Containment is measured separately by the automation-rate metric; quality must not double-penalize it.) 5. **Judge only what the assistant could see.** Cold session, no account, no order history. Never penalize the assistant for not knowing what a logged-out first-time visitor couldn't provide. Before failing a check, apply the hindsight self-check: **given only what the assistant could see at that turn, was this actually the wrong behavior?** 6. **Lane standards.** Shopping turns are judged by sales standards (proactive recommendation is good), support turns by support standards. Never mark proactive selling as a failure in the shopping lane. 7. **Real failures only.** A failed check must correspond to an articulable, concrete miss — "the reply ignored the sizing question" — not a style preference. Transcripts contain widget chrome (cookie banners, quick-reply chip labels, timestamps): judge the substance and ignore the chrome.

Shopping lane — 5 dimensions, /100 (v2.3)

answer (max 30)

check	pts	passes when
a_direct	14	each shopper question gets a direct, on-topic, substantive response. A generic greeting or brand boilerplate ("Hi! How can I help?", "We're so happy to have you here!") that does not actually answer the question fails — warmth, enthusiasm and emojis are not substance.
a_consistent	9	no contradiction across turns; no invented policies/specs
a_no_ignored	7	no shopper turn is left unanswered or answered with an unrelated reply

discovery (max 20) — the PMF core of a shopping assistant: does it *understand the shopper* before selling? Judge relative to how broad the ask is (our `beginner` / `gift` / `problem-solver` themes open vague; `everyday-value` / `compare-budget` open specific).

check	pts	passes when
d_clarify	8	on a BROAD/ambiguous opener ("a gift", "where do I start", "something for X") it asks a relevant clarifying question (budget, recipient, use-case, skin type, size, preference) before recommending. On an already-specific ask, going straight to a targeted answer also passes — the fail is <i>guessing blind on a vague ask</i> .
d_progressive	7	it uses the shopper's answers to narrow/refine across turns (builds a profile) — a later recommendation reflects earlier stated constraints, not a generic pick repeated.

check	pts	passes when
d_not_dump	5	it does not dump a generic best-seller list on a vague opener before understanding needs (a wall of unqualified products fails).

recommendation (max 22)

check	pts	passes when
r_named	9	recommends at least one specific, named product
r_fit	8	ties the recommendation to the shopper's specific stated constraints (budget, use-case, skin type, recipient...) with real reasoning. A product dropped with generic praise/emojis but no rationale connected to what the shopper actually said fails .
r_plausible	5	recommendations are appropriate to the store's catalog and the request

rich (max 18) — every check is **capped by deterministic signals** (see `signals`)

check	pts	signal gate	passes when
e_price	6	has_price	a concrete price is attached to a recommended product
e_link	7	has_link	a product link/card the shopper can open is presented
e_reviews	3	has_reviews	review counts/ratings (or images) support the recommendation
e_options	2	has_options	multiple distinct options are laid out for comparison

close (max 10)

check	pts	passes when
c_cta	5	ends with a concrete next step (open the product, pick a size, apply code...)
c_cart	3	facilitates purchase mechanics (add-to-cart, checkout guidance). If the shopper explicitly asks to add to cart or for a total/checkout and the agent does not actually do it (re-asks, deflects, or loops), this fails — an unfulfilled explicit purchase request is a hard miss.
c_clean	2	conversation closes cleanly (no dangling question, no mid-thought stop)

Calibration note (v2.2): added the **discovery** dimension (20 pts) — asking smart clarifying questions on broad asks and building a profile is a core Shopping-Assistant PMF signal the v2.1 rubric under-weighted (a lone 5-pt `a_clarify`). Points came from rich (25→18, product-card spraying was over-rewarded), recommendation (25→22), close (15→10), and folding `a_clarify` into `d_clarify` . Support lane unchanged.

>

Calibration note (v2.3): tightened three shopping checks after reading transcripts where a fluffy, emoji-heavy agent scored ~90+ by mechanically ticking boxes (a product was named, a price shown) while (a) opening with a generic greeting instead of answering, and (b) never completing an explicitly-requested cart/total. **Substance over style** — judge what the shopper could actually DO with the answer, not how warm it sounds. No point weights changed; the pass bars on `a_direct` (no boilerplate credit), `r_fit` (rationale must tie to stated constraints) and `c_cart` (unfulfilled explicit purchase request = fail) are stricter. Applies to every vendor.

Support lane — 4 dimensions, /100

resolution (max 40)

check	pts	passes when
s_answered	18	the PRIMARY ask gets a complete, store-specific answer or procedure in-channel. A generic pointer ("check our returns page"), a partial answer, or industry-general boilerplate fails
s_outcome	12	the ending is outcome-correct: resolved in-channel, or a justified handover done well (context collected first, expectations set). An unjustified bail on an answerable question fails
s_no_deflect	10	doesn't push the shopper out of channel ("email us", "call us") when the question was answerable in-chat

accuracy (max 25)

check	pts	passes when
g_specific	13	gives ≥2 concrete, store-specific facts (a number, timeframe, named policy term, concrete condition). One vague reassurance ("we'll sort it out") fails
g_consistent	5	no self-contradiction across turns
g_grounded	7	policy/product claims read as grounded in THIS store (named policies, actual conditions, store-specific procedures) rather than plausible industry generalities

actionability (max 20)

check	pts	passes when
t_steps	12	the shopper leaves with steps they can execute NOW in their situation (cold session, no account) — numbered or clearly sequenced; "reach out if..." alone fails
t_complete	8	the answer covers the actual ask (not a fragment of it)

close (max 15)

check	pts	passes when
k_expectations	8	sets expectations with WHO acts and WHEN (a timeframe) — vague "soon"/"we'll be in touch" fails
k_clean	7	clean, complete close

Calibration note (v2.1): the five support checks above were tightened after the first pinned-cohort run showed them passing at 83–100% (non-discriminating bars); `g_consistent` was split into `g_consistent` (5) + `g_grounded` (7). Weights per dimension are unchanged.

Output contract (per conversation)

```
{ "k": "<opaque key from the batch>",  
  "mode": "shopping|support",  
  "checks": { "<check_id>": { "pass": true, "evidence": "<short verbatim quote>" }, ... },  
  "resolution_class": "resolved|partial|deflected|failed",  
  "learning": "<one concise sentence – the standout strength or gap>" }
```

- Every check in the lane's table MUST appear. `pass:true` without a non-empty `evidence`

quote is invalid and the check is treated as failed at merge.

- `resolution_class`: `resolved` = handled fully in-channel; `partial` = some substance but

incomplete; `deflected` = pushed out of channel; `failed` = no meaningful help. A justified, well-executed handover with expectations set is `partial`, not `failed`.

- Sub-scores and the /100 total are computed at merge from the check booleans; deterministic

signal gates are re-derived from the stored transcript and enforced there.

Audit loop

After each judging run, `eval-audit.js` samples scored conversations for an adversarial second pass: the auditor re-reads the transcript and classifies each verdict `AGREE` / `FALSE_POSITIVE` (credited without real evidence) / `FALSE_NEGATIVE` (failed on a judge trap — see `notes/judge-traps.md`). A run is marked **trusted** when agreement $\geq 90\%$; per-check accuracy is reported next to the vendor scores. Verdicts a judge got wrong are re-scored, not silently kept.